

LECTURE 6

Neural Networks: Hidden Layers & Backpropagation

Part 1: why hidden layers — one neuron's straight line, the XOR wall, non-linear activations and learned feature maps
Part 2: backpropagation by hand — one forward and one backward pass through a 2–2–1 sigmoid network

AK DHAMIJA, Researcher & Instructor

AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Learning Objectives & Session Structure

By the end of this lecture you will be able to

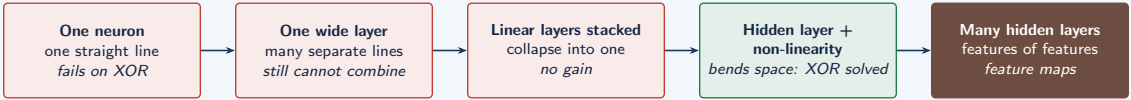
1. Show that one neuron draws one straight boundary, and prove it cannot compute XOR
2. Show that wide-but-linear layers, or stacked linear layers, add no power
3. Solve XOR by hand with one ReLU hidden layer; explain hidden layers as *learned feature maps*
4. State the universal approximation theorem, its fine print, and why depth still matters
5. Run a full forward pass $z^{(1)} \to a^{(1)} \to z^{(2)} \to a^{(2)} \to L$ in a 2–2–1 sigmoid network
6. Derive $\delta^{(2)} = a^{(2)} - y$ and back-propagate it with the chain rule
7. Compute every gradient and one gradient-descent update, and check its effect

Session Structure (45 min)

- **5 min** — Opening Q&A: recap of Lectures 1–5
- **13 min** — **Part 1, Why hidden layers?** One neuron, the XOR wall, non-linearity, XOR solved, feature maps
- **17 min** — **Part 2, Backpropagation by hand:** set-up, forward pass, backward pass, update and check
- **10 min** — Auto-quiz (15 questions)

Where this leads: every network in the rest of the course — the CNN (L7), the transformer (L8–L9), the DQN and PPO agents (L10–L11) — has hidden layers with non-linear activations, and every one is trained by the backpropagation of Part 2.

The Story in One Slide



The limitation

A neuron computes a weighted sum, so its boundary is a straight line (a plane in higher dimensions).

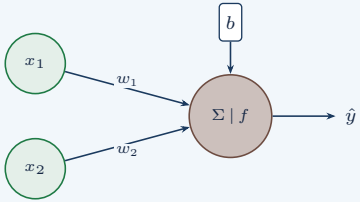
The fix

Put a *non-linear* activation between layers; the hidden layer re-maps the data into new coordinates.

The bonus

Those new coordinates are *features* the network invents for itself — no manual feature engineering.

A Single Neuron Draws a Straight Line

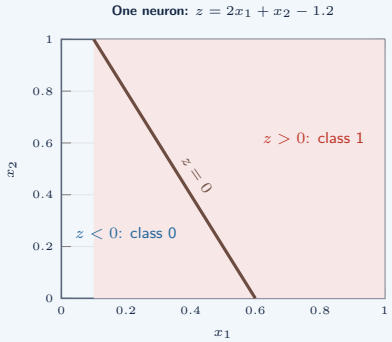


DECISION BOUNDARY

$$w_1 x_1 + w_2 x_2 + b = 0 \quad \Longleftrightarrow \quad x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$$

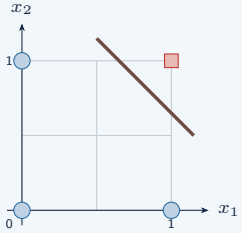
Whatever the activation

Step, sigmoid or ReLU are all *monotonic* in $z = w \cdot x + b$, so "output above threshold" is the same as "z above some constant". The boundary is always a straight line — in d dimensions, a flat hyperplane.



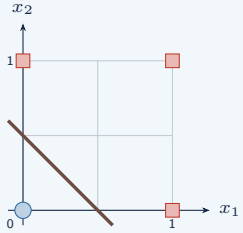
Training a single neuron only *moves and rotates* this line. It can never bend it.

What One Neuron Can Do: AND and OR



AND: $w = (1, 1),\ b = -1.5$

x_1	x_2	z	$\hat{y}$
0	0	-1.5	0
0	1	-0.5	0
1	0	-0.5	0
1	1	0.5	**1**



OR: $w = (1, 1),\ b = -0.5$

x_1	x_2	z	$\hat{y}$
0	0	-0.5	0
0	1	0.5	**1**
1	0	0.5	**1**
1	1	1.5	**1**

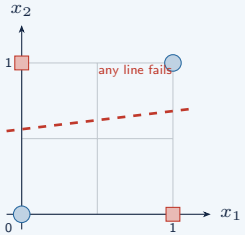
Rule: step output

$\hat{y} = 1$ if $z = w_1 x_1 + w_2 x_2 + b > 0$, else 0. Both problems are **linearly separable**: one line puts all squares on one side and all circles on the other.

Rosenblatt's perceptron (1958) learned exactly such lines, and it was hailed as the start of machine intelligence.

Legend: ○ = class 0, □ = class 1.

The XOR Wall: A Problem No Single Neuron Can Solve



XOR: output 1 when the inputs *differ*. The squares sit on one diagonal, the circles on the other.

Proof by contradiction

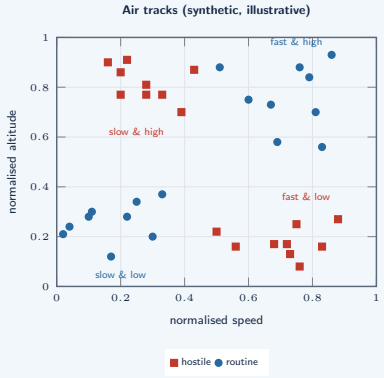
Suppose some w_1, w_2, b worked (output 1 iff $z > 0$). The four rows of the truth table demand:

$$\begin{aligned} (0,0) \to 0: &\quad b \le 0 && (1)\\ (1,0) \to 1: &\quad w_1 + b > 0 && (2)\\ (0,1) \to 1: &\quad w_2 + b > 0 && (3)\\ (1,1) \to 0: &\quad w_1 + w_2 + b \le 0 && (4) \end{aligned}$$

Add (2) and (3): $w_1 + w_2 + 2b > 0$, so $w_1 + w_2 + b > -b \ge 0$ using (1). This contradicts (4). ■

History: Minsky and Papert's book *Perceptrons* (1969) highlighted this limitation; it contributed to a long slump in neural-network research until multi-layer training (backpropagation) was popularised in 1986.

XOR Is Not a Toy: Non-Separable Patterns in Real Data



An XOR-shaped threat picture (illustrative)

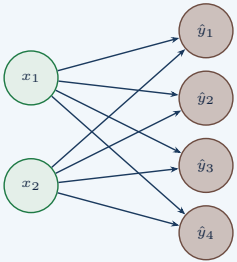
- **Fast & low**: sea-skimming or cruise-missile profile — hostile
- **Slow & high**: loitering surveillance platform — hostile
- **Slow & low**: own helicopters — routine
- **Fast & high**: airliners, own jets — routine

Neither speed nor altitude alone separates the classes; only their *combination* does — the XOR pattern.

A single neuron (logistic regression) on these two inputs can do little better than guessing the majority class.

Other common non-separable shapes: a target inside a ring of clutter, interleaved spirals, "either A or B but not both" rules.

One Layer with Many Neurons Is Still Linear



4 outputs = 4 independent lines

A SINGLE LAYER

$$\hat{y}_k \;=\; f\big(w_k \cdot x + b_k\big), \quad k \;=\; 1, \ldots, K$$

No neuron sees another neuron's answer. Each output draws its own line on the *same raw inputs*; nothing combines them. For XOR, every one of the K neurons faces the same impossibility proof.

Multi-class is also linear

With softmax over K outputs (multinomial logistic regression), class k wins where $w_k \cdot x + b_k$ is largest. The regions are separated by straight lines, and every region is *convex* — no class can occupy two opposite corners.

Width alone adds more lines; it does not let the network *reason with* those lines ("inside line 1 AND outside line 2"). That needs a second stage that takes the lines as its inputs — a hidden layer.

Stacking Linear Layers Collapses into One

The algebra

Two layers with *no* activation (identity f):

$$\hat{y} = W_2(W_1 x + b_1) + b_2 = \underbrace{(W_2 W_1)}_{W} x + \underbrace{(W_2 b_1 + b_2)}_{b}$$

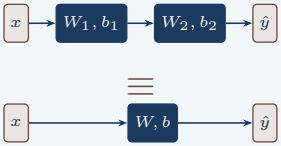
A product of matrices is just another matrix, so 2 — or 100 — linear layers equal **one** linear layer.

Numerical check

$$W_1 = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix},\ b_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix},\ W_2 = (1 \quad -1),\ b_2 = 0.5$$

Collapsed: $W = W_2 W_1 = (1,\ 1),\ b = W_2 b_1 + b_2 = 1.5$

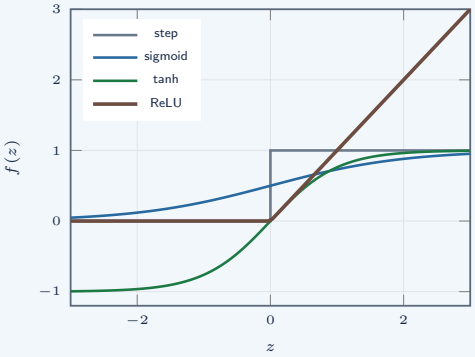
At $x = (1, 1)$: two layers $\rightarrow h = (4, 1)$, $\hat{y} = 4 - 1 + 0.5 = \mathbf{3.5}$; one layer $\rightarrow 1 + 1 + 1.5 = \mathbf{3.5}$.
At $x = (2, -1)$: both give **2.5**.



Conclusion: depth without non-linearity is an illusion. The hidden layer must *bend* its output, or the network is still a single straight line.

The same argument shows why every CNN conv layer and every Transformer feed-forward block is followed by a non-linear activation.

Non-Linear Activation Functions



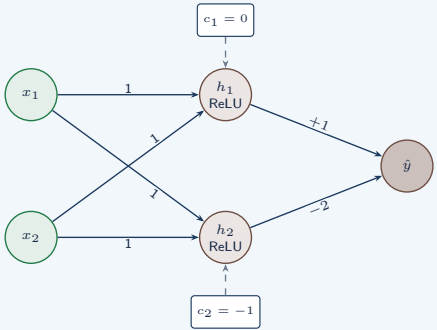
Name	$f(z)$	Notes
Step	1 if $z > 0$, else 0	no usable gradient
Sigmoid	$1/(1+e^{-z})$	range $(0,1)$; $f' \leq 0.25$
Tanh	$\tanh z$	range $(-1,1)$
ReLU	$\max(0,z)$	default; $f' = 1$ if $z > 0$

What a non-linearity must do

Be *not* of the form $az + b$, so that composition no longer collapses. Even the very simple ReLU — two straight pieces joined at a kink — is enough.

ReLU is preferred in hidden layers: cheap, and its gradient does not shrink for active units (contrast the sigmoid's vanishing gradient in Part 2 of this lecture).

A Hidden Layer Solves XOR



THE NETWORK

$$h = \text{ReLU}(Wx + c), \quad \hat{y} = w^{\mathsf{T}} h$$

$$W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \; c = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \; w = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Reading the hidden units

- $h_1 = \text{ReLU}(x_1 + x_2)$ counts *how many* inputs are on — an "OR-ish" detector
- $h_2 = \text{ReLU}(x_1 + x_2 - 1)$ is non-zero only when *both* are on — an "AND" detector
- Output $= h_1 - 2h_2$: "at least one, but subtract heavily if both"

XOR = OR AND NOT AND. The hidden layer computes the two pieces; the output combines them — exactly the combining that a single layer could not do.

This is the classic textbook solution (Goodfellow, Bengio & Courville, *Deep Learning*, 2016, §6.1). In practice these weights would be *learned* by backpropagation.

XOR Solved: The Forward Pass for All Four Inputs

x_1	x_2	$z_1 = x_1 + x_2$	$z_2 = x_1 + x_2 - 1$	$h_1 = \mathrm{ReLU}(z_1)$	$h_2 = \mathrm{ReLU}(z_2)$	$\hat{y} = h_1 - 2h_2$	XOR
0	0	0	-1	0	**0**	$0 - 0 = \mathbf{0}$	0
0	1	1	0	1	0	$1 - 0 = \mathbf{1}$	1
1	0	1	0	1	0	$1 - 0 = \mathbf{1}$	1
1	1	2	1	2	1	$2 - 2 = \mathbf{0}$	0

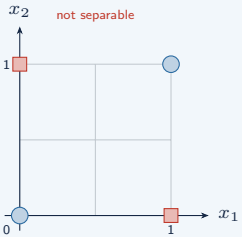
Where ReLU matters

Row 1: $z_2 = -1$ is clipped to 0. Without the clip, $h_2 = -1$ and $\hat{y} = 0 - 2(-1) = 2$ — wrong.

Check linear version

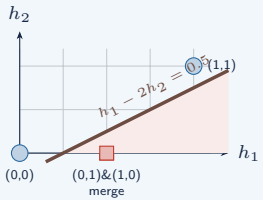
Drop ReLU: $\hat{y} = z_1 - 2z_2 = -x_1 - x_2 + 2$ — a single line, outputs $2, 1, 1, 0$. Fails, as the collapse slide predicted.

All four correct. Two hidden ReLU units and three output weights solve what no single neuron, and no number of linear layers, can.



Input space (x_1, x_2)

h



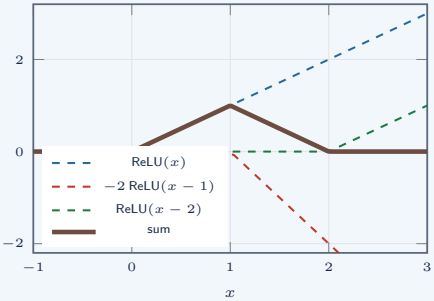
Hidden space (h_1, h_2)

In the hidden coordinates the two class-1 points land on the *same spot*, and one straight line now separates the classes.

Hidden activations are a learned feature map

The hidden layer is a function $x \mapsto h(x)$ that sends each input to a new point. The output neuron is still just a line — but a line *in the new space*. Training chooses the map so that the final problem becomes easy (linearly separable). This is **representation learning**: the network invents the features ("how many on?", "both on?") instead of an engineer hand-crafting them.

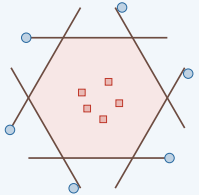
Each ReLU Unit Adds a Bend: Building Any Shape



THREE HIDDEN UNITS MAKE A BUMP

$$f(x) = \mathrm{ReLU}(x) - 2\,\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2)$$

Values:
$f(0) = 0,\ f(0.5) = 0.5,\ f(1) = 1,\ f(1.5) = 0.5,\ f(2) = 0$, and 0 beyond.



6 hidden units = 6 lines; output fires inside all six

In 2-D

Each hidden unit contributes one line; the output combines them ("inside line 1 AND line 2 AND ..."). Six units enclose a target inside a ring of clutter; more units give a smoother region.

UNIVERSAL APPROXIMATION (CYBENKO 1989; HORNIK 1991)

A network with **one hidden layer** of enough non-linear units can approximate any continuous function on a bounded region to any desired accuracy.

Why it is plausible

The previous slide: every ReLU adds a kink; enough bumps, scaled and shifted, can trace out any smooth curve — like building a curve from many short straight segments.

So the answer to "*can* a network represent my problem?" is almost always yes — *if* it has a hidden layer with a non-linear activation.

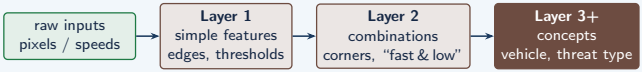
Fine print 1: how many units? The theorem says enough units *exist*; for complex functions one layer may need astronomically many.

Fine print 2: can we find them? It says nothing about whether training will *find* the right weights.

Fine print 3: will it generalise? Fitting the training data is not the same as predicting new data well.

These gaps are why practice moved from *wide and shallow* to *deep*: depth reaches the same functions far more economically.

Why Depth: Features of Features



Composition

A deep network computes $f_3(f_2(f_1(x)))$. Each layer re-uses everything below it, so a feature learned once (an edge, a "low-altitude" detector) serves many higher features.

Efficiency

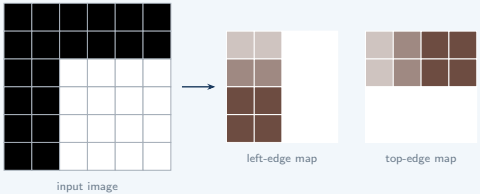
Theory and experiment agree that the number of distinct linear regions a ReLU network can carve grows *exponentially* with depth but only polynomially with width (Montúfar et al., 2014). Depth buys complexity cheaply.

Analogy: a staff process. Sensor operators report raw contacts; analysts group contacts into formations; the intelligence officer infers intent. No single person does it all, and each level builds on the one below.

Cost of depth: deeper networks are harder to train (vanishing gradients). ReLU activations, careful initialisation, batch normalisation and residual connections are the tools that made 100-layer networks practical.

What a feature map is

The set of activations produced by one hidden unit (or one filter) across the input. In a dense layer it is one number per example; in a CNN it is a whole *image* — the filter's response at every location.



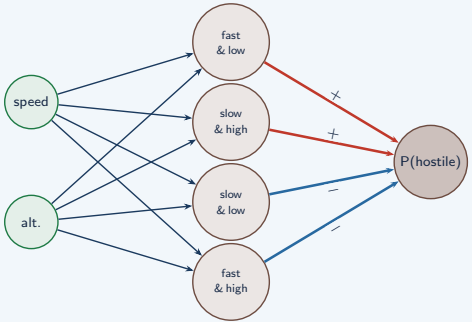
Two hidden filters from Lecture 7 (CNNs) produce two feature maps of the same image.

Advantages

- **Automatic feature engineering**: the network discovers useful intermediate quantities from labels alone
- **Re-usable**: the same features serve many outputs and tasks (transfer learning)
- **Compact**: a few hundred features summarise thousands of raw inputs
- **Inspectable**: maps can be visualised to see *what* the network detects
- **Hierarchical**: deeper maps combine shallower ones into parts and objects

In the XOR network, h_1 ("how many on") and h_2 ("both on") are the feature maps; in a CNN they are edge and texture maps; in a Transformer they are the context-aware token vectors.

Defence Example: Hidden Units as Learned Indicators



An idealised picture of what a trained hidden layer could learn on the air-track data.

How the network solves the threat picture

Each hidden unit becomes an *indicator* that fires for one region of the speed–altitude plane. The output neuron then needs only a weighted vote: "+" for the two hostile profiles, "−" for the two routine ones — a linear decision in the indicator space.

Reality check: real hidden units rarely align with neat human labels; their features are often distributed across many units. Interpreting them needs care — and a human analyst for high-consequence decisions.

Add more inputs (RCS, heading change rate, IFF response) and the hidden layer learns combinations no analyst has written down.

"More neurons in one layer solve everything." Without a hidden layer, each output is still one straight line, however many outputs there are.

"More layers always help." Only if separated by non-linear activations — and only if the network can still be trained.

"The activation is a detail." It is the *entire* source of the network's power beyond linear regression.

"Hidden features are designed by engineers." They are learned by backpropagation from the labels alone.

"Universal approximation means any network will work." It guarantees existence, not size, trainability or generalisation.

Self-check: reproduce the XOR forward-pass table, and explain in one sentence why the ReLU in row 1 is essential.

Part 1 Summary and Quick Check — Why Hidden Layers?

ONE NEURON

$\hat{y} = f(w \cdot x + b)$: one straight boundary $w \cdot x + b = 0$

LINEAR STACKING COLLAPSES

$$W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2)$$

HIDDEN LAYER

$$h = f(W_1 x + b_1), \quad \hat{y} = W_2 h + b_2, \quad f \text{ non-linear}$$

XOR BY HAND

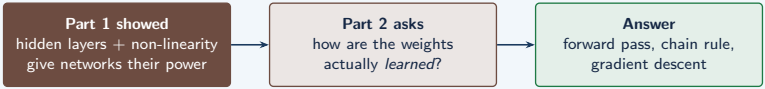
$$\hat{y} = \mathrm{ReLU}(x_1 + x_2) - 2\,\mathrm{ReLU}(x_1 + x_2 - 1)$$

FEATURE MAP VIEW

hidden layer = learned map $x \mapsto h$ making the problem linear

Quick check

1. Give weights and bias for a single neuron computing NAND.
2. Which inequality in the XOR proof uses the $(0, 0)$ row, and why is it needed?
3. Collapse $W_1 = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}$, $b_1 = 0$, $W_2 = (1 \;\; 1)$, $b_2 = -1$ into one layer.
4. In the XOR network, what does $\hat{y}$ become for $(0, 0)$ if ReLU is replaced by the identity?
5. Name two advantages of learned feature maps over hand-engineered features.

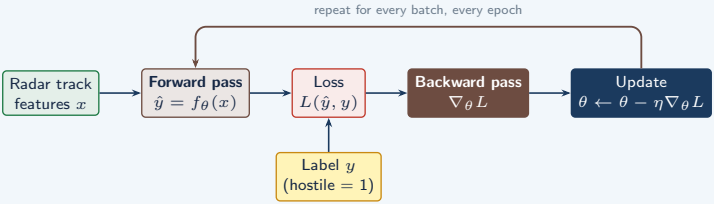


The plan

One training example, a 2–2–1 network with sigmoid units and binary cross-entropy loss. We compute *every* number: five forward steps (F1–F5), five backward steps (B1–B5), all nine gradients, then one update with learning rate $\eta = 0.5$.

Keep the reference text open: each step on the slides has a matching worked derivation and a "check your understanding" box.

Where Backpropagation Sits in a Defence ML Pipeline



L2 Classification

A neural threat classifier outputs $P(\text{hostile} \mid x)$ through a sigmoid — today's output unit.

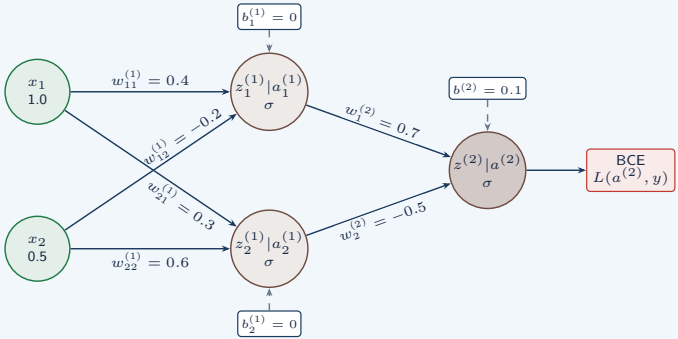
L10 Deep Q-Networks

DQN minimises a TD loss; the gradient reaches every layer by the same chain rule.

L11 Policy Gradients

$\nabla_\theta \log \pi_\theta(a|s)$ is computed by back-propagation through the policy network.

The Network and Its Parameters



INPUT AND TARGET

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix}, \quad y = 1$$

HIDDEN LAYER (1)

$$W^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{pmatrix} = \begin{pmatrix} 0.4 & -0.2 \\ 0.3 & 0.6 \end{pmatrix}$$

$$b^{(1)} = (b_1^{(1)}, b_2^{(1)}) = (0, 0)$$

OUTPUT LAYER (2)

$$w^{(2)} = (w_1^{(2)}, w_2^{(2)}) = (0.7, -0.5)$$

$$b^{(2)} = 0.1$$

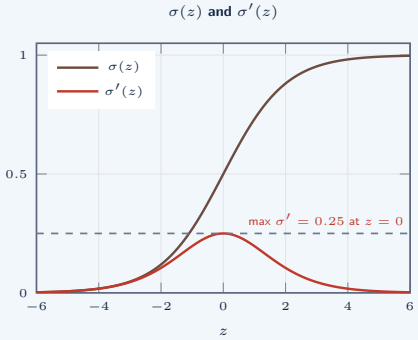
Notation and the Sigmoid Activation

Superscript = layer Subscript = index

- $w_{jk}^{(1)}$: weight from input x_k to hidden unit j (row j, column k of $W^{(1)}$)
- $b_j^{(1)}$, $z_j^{(1)}$, $a_j^{(1)}$: bias, pre-activation, activation of hidden unit j
- $w_j^{(2)}$: weight from hidden unit j to the output
- $z^{(2)}$, $a^{(2)} = \hat{y}$: output pre-activation and prediction
- $\delta_j^{(l)} = \partial L / \partial z_j^{(l)}$: error signal at unit j of layer l

SIGMOID AND ITS DERIVATIVE

$$\sigma(z) \;=\; \frac{1}{1+e^{-z}}, \qquad \sigma'(z) \;=\; \sigma(z)\big(1-\sigma(z)\big)$$



$\sigma'(z) \le 0.25$ everywhere: each sigmoid layer shrinks the gradient by at least $4\times$ — the root of the **vanishing-gradient** problem.

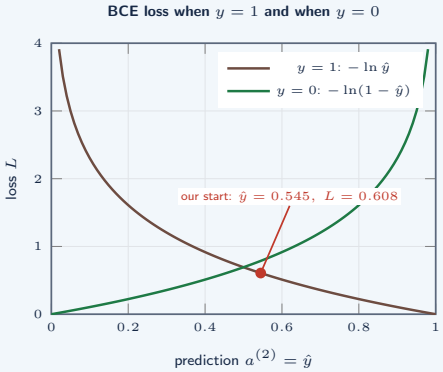
The Loss Function: Binary Cross-Entropy

BINARY CROSS-ENTROPY (BCE)

$$L \;=\; -\big[\, y \ln a^{(2)} \;+\; (1 \,-\, y) \ln\!\big(1 \,-\, a^{(2)}\big)\big]$$

- $y = 1$ (*hostile*): $L = -\ln a^{(2)}$ — small when $\hat{y} \to 1$
- $y = 0$ (*friendly*): $L = -\ln(1 - a^{(2)})$ — small when $\hat{y} \to 0$
- Confident wrong answers are punished heavily: $-\ln(0.01) = 4.61$
- It is the negative log-likelihood of a Bernoulli label — the natural loss for a sigmoid output

Defence reading: a missile-warning classifier that says "1% chance hostile" for a real threat pays a loss of 4.61 — 9× the loss of an undecided 50% call.



Unit by unit

$$
\begin{aligned}
z_1^{(1)} &= w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + b_1^{(1)} \\
&= (0.4)(1.0) + (-0.2)(0.5) + 0 = 0.4 - 0.1 = \mathbf{0.3000} \\
z_2^{(1)} &= w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + b_2^{(1)} \\
&= (0.3)(1.0) + (0.6)(0.5) + 0 = 0.3 + 0.3 = \mathbf{0.6000}
\end{aligned}
$$

Matrix form

$$
z^{(1)} = \begin{pmatrix} 0.4 & -0.2 \\ 0.3 & 0.6 \end{pmatrix} \begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.3 \\ 0.6 \end{pmatrix}
$$

Reading the indices

$w_{12}^{(1)}$ = layer 1, *into* hidden unit 1, *from* input 2. Row j of $W^{(1)}$ collects every weight entering hidden unit j, so one row times x gives one $z_j^{(1)}$.

The negative weight $w_{12}^{(1)} = -0.2$ means input x_2 *inhibits* hidden unit 1 — e.g. a high IFF-response score lowering a "threat" feature detector.

Forward Pass F2 — Hidden Activations $a_j^{(1)} = \sigma(z_j^{(1)})$

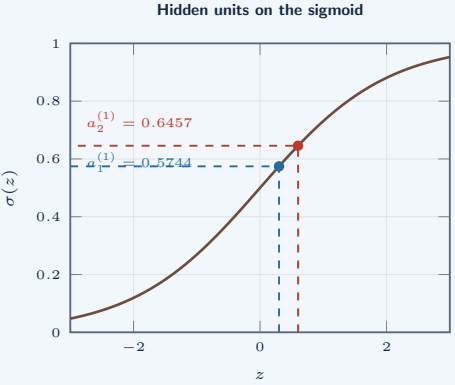
$$a_1^{(1)} = \sigma(0.3) = \frac{1}{1+e^{-0.3}} = \frac{1}{1+0.740818} = \mathbf{0.574443}$$

$$a_2^{(1)} = \sigma(0.6) = \frac{1}{1+e^{-0.6}} = \frac{1}{1+0.548812} = \mathbf{0.645656}$$

What the sigmoid does

Squashes any real z into $(0, 1)$. Near $z = 0$ it is almost linear ($\sigma(z) \approx 0.5 + z/4$): check $0.5 + 0.3/4 = 0.575$ ✓. Far from 0 it saturates.

Both hidden units sit in the near-linear region, so gradients will flow well — this network is easy to train.

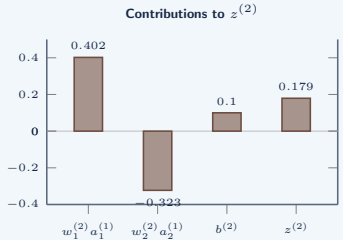


F3: pre-activation $z^{(2)} = w^{(2)}a^{(1)} + b^{(2)}$

$$\begin{aligned} z^{(2)} &= w_1^{(2)}a_1^{(1)} + w_2^{(2)}a_2^{(1)} + b^{(2)} \\ &= (0.7)(0.574443) + (-0.5)(0.645656) + 0.1 \\ &= 0.402110 - 0.322828 + 0.1 = \mathbf{0.179282} \end{aligned}$$

F4: prediction $a^{(2)} = \hat{y} = \sigma(z^{(2)})$

$$a^{(2)} = \frac{1}{1 + e^{-0.179282}} = \frac{1}{1 + 0.835871} = \mathbf{0.544701}$$



Hidden unit 2 *pulls the output down*: its positive activation meets a negative weight. Hold this thought — it explains a sign in the backward pass.

F5: binary cross-entropy

Since $y = 1$, the $(1 - y)$ term vanishes:

$$L = -\ln(0.544701) = \mathbf{0.607519}$$

Forward-pass results

$z_1^{(1)}$	0.3000	$a_1^{(1)}$	0.574443
$z_2^{(1)}$	0.6000	$a_2^{(1)}$	0.645656
$z^{(2)}$	0.179282	$a^{(2)} = \hat{y}$	0.544701
Loss $L = 0.607519$			

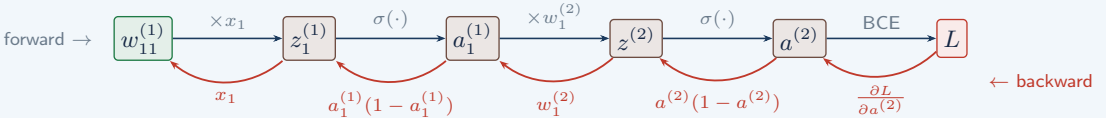
Interpretation: the network is barely better than a coin-flip (54.5%) on a target it should call "hostile". The backward pass tells each parameter how to move to raise that probability.

Save for the backward pass

- x_1, x_2 (multiply into $\partial L/\partial w^{(1)}$)
- $a_1^{(1)}, a_2^{(1)}$ (multiply into $\partial L/\partial w^{(2)}$, and give σ')
- $a^{(2)}$ (gives $\delta^{(2)}$)

Frameworks such as PyTorch store exactly these *activations* in memory during the forward pass.

The Chain Rule as a Computational Graph



CHAIN RULE ALONG THE PATH

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \underbrace{\frac{\partial L}{\partial a^{(2)}}\frac{\partial a^{(2)}}{\partial z^{(2)}}}_{\delta^{(2)}} \frac{\partial z^{(2)}}{\partial a_1^{(1)}} \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}} = \underbrace{\delta^{(2)}\, w_1^{(2)}\, \sigma'(z_1^{(1)})}_{\delta_1^{(1)}}\, x_1$$

Backpropagation = the chain rule + reuse. Compute $\delta^{(2)}$ once, reuse it for every output-layer gradient; compute each $\delta_j^{(1)}$ once, reuse it for every weight entering unit j.

Backward Pass B1 — Output Error Signal $\delta^{(2)}$

Two local derivatives

$$\frac{\partial L}{\partial a^{(2)}} = -\frac{y}{a^{(2)}} + \frac{1-y}{1-a^{(2)}} = \frac{a^{(2)} - y}{a^{(2)}\left(1-a^{(2)}\right)}$$

$$\frac{\partial a^{(2)}}{\partial z^{(2)}} = \sigma'(z^{(2)}) = a^{(2)}\left(1-a^{(2)}\right)$$

Multiply: the $a^{(2)}(1-a^{(2)})$ factors cancel

$$\delta^{(2)} = \frac{\partial L}{\partial z^{(2)}} = a^{(2)} - y = 0.544701 - 1 = \textbf{-0.455299}$$

The sigmoid + BCE identity

$\delta^{(2)} = \hat{y} - y$: *prediction minus target*. No σ' factor at the output, so the output layer never saturates its own gradient. This is why BCE, not squared error, is paired with a sigmoid output.

With squared error $L = \frac{1}{2}(\hat{y} - y)^2$ we would get $\delta^{(2)} = (\hat{y} - y)\,\hat{y}(1-\hat{y}) = -0.1129$ — four times weaker here, and near zero when the network is confidently wrong.

Since $z^{(2)} = w_1^{(2)} a_1^{(1)} + w_2^{(2)} a_2^{(1)} + b^{(2)}$:

$$\frac{\partial z^{(2)}}{\partial w_j^{(2)}} = a_j^{(1)}, \quad \frac{\partial z^{(2)}}{\partial b^{(2)}} = 1$$

$$\frac{\partial L}{\partial w_1^{(2)}} = \delta^{(2)} a_1^{(1)} = (-0.455299)(0.574443) = \textbf{-0.261543}$$

$$\frac{\partial L}{\partial w_2^{(2)}} = \delta^{(2)} a_2^{(1)} = (-0.455299)(0.645656) = \textbf{-0.293967}$$

$$\frac{\partial L}{\partial b^{(2)}} = \delta^{(2)} = \textbf{-0.455299}$$

Gradient = error × input

Every weight gradient is **(error signal at the receiving unit)** × **(activation on the sending side)**. A bias has input 1, so its gradient is the error signal itself.

All three gradients are negative, so gradient *descent* will *increase* $w_1^{(2)}$, $w_2^{(2)}$ and $b^{(2)}$ — each increase raises $z^{(2)}$ and pushes $\hat{y}$ toward 1.

B3: sigmoid slopes at the hidden units

$$\sigma'(z_1^{(1)}) = a_1^{(1)}(1 - a_1^{(1)}) = (0.574443)(0.425557) = \mathbf{0.244458}$$
$$\sigma'(z_2^{(1)}) = a_2^{(1)}(1 - a_2^{(1)}) = (0.645656)(0.354344) = \mathbf{0.228784}$$

B4: back-propagate $\delta_j^{(1)} = \delta^{(2)}\, w_j^{(2)}\, \sigma'(z_j^{(1)})$

$$\delta_1^{(1)} = (-0.455299)(0.7)(0.244458) = \mathbf{-0.077911}$$
$$\delta_2^{(1)} = (-0.455299)(-0.5)(0.228784) = \mathbf{0.052083}$$

Why is $\delta_2^{(1)}$ positive? $w_2^{(2)} < 0$, so raising $a_2^{(1)}$ *lowers* $\hat{y}$ — the wrong direction for a hostile target. The gradient tells unit 2 to quieten down.

Shrinking signal

$|\delta^{(2)}| = 0.455$ but $|\delta_j^{(1)}| \leq 0.078$: one weight ($< 1$) and one σ' (≤ 0.25) cut the signal about $6\times$. Ten such layers would cut it by $\sim 10^8$.

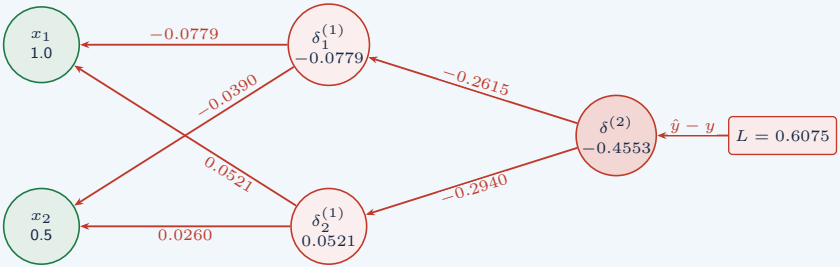
Rules: $\dfrac{\partial L}{\partial w_{jk}^{(1)}} = \delta_j^{(1)}\, x_k, \qquad \dfrac{\partial L}{\partial b_j^{(1)}} = \delta_j^{(1)}$

	Calculation	Gradient		Calculation	Gradient
$\partial L/\partial w_{11}^{(1)}$	$(-0.077911)(1.0)$	**-0.077911**	$\partial L/\partial w_{21}^{(1)}$	$(0.052083)(1.0)$	**0.052083**
$\partial L/\partial w_{12}^{(1)}$	$(-0.077911)(0.5)$	**-0.038956**	$\partial L/\partial w_{22}^{(1)}$	$(0.052083)(0.5)$	**0.026041**
$\partial L/\partial b_1^{(1)}$	$\delta_1^{(1)}$	**-0.077911**	$\partial L/\partial b_2^{(1)}$	$\delta_2^{(1)}$	**0.052083**

MATRIX FORM: ONE OUTER PRODUCT

$$\nabla_{W^{(1)}} L \;=\; \delta^{(1)} x^{\mathsf{T}} \;=\; \begin{pmatrix} -0.077911 \\ 0.052083 \end{pmatrix} \begin{pmatrix} 1.0 & 0.5 \end{pmatrix} \;=\; \begin{pmatrix} -0.077911 & -0.038956 \\ 0.052083 & 0.026041 \end{pmatrix}$$

Backward-Flow Diagram

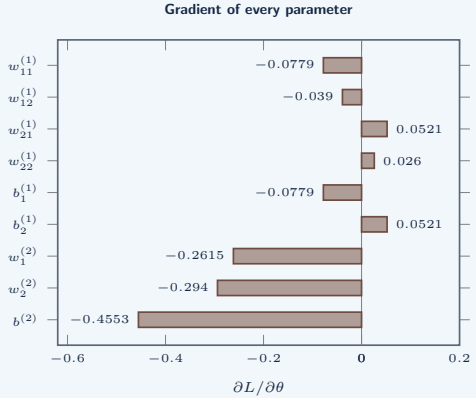


How to read it

Nodes carry $\delta = \partial L / \partial z$. Edges carry $\partial L / \partial w$ for that connection = (node δ on the right) $\times$ (activation on the left).

Information now flows *right to left*: the same graph, traversed backwards, delivers every gradient in one sweep.

All Gradients at a Glance



What the signs say

- **Negative** gradient $\Rightarrow$ descent *increases* the parameter
- **Positive** gradient $\Rightarrow$ descent *decreases* the parameter
- Hidden unit 1 (feeds $+0.7$): all its incoming weights grow
- Hidden unit 2 (feeds -0.5): all its incoming weights shrink

Output-layer gradients are 3–6$\times$ larger than hidden-layer ones: the output layer learns fastest at first.

One Gradient-Descent Step ($\eta = 0.5$, assumed) and a Sanity Check

UPDATE RULE

$$\theta \quad \leftarrow \quad \theta \; - \; \eta \, \frac{\partial L}{\partial \theta}$$

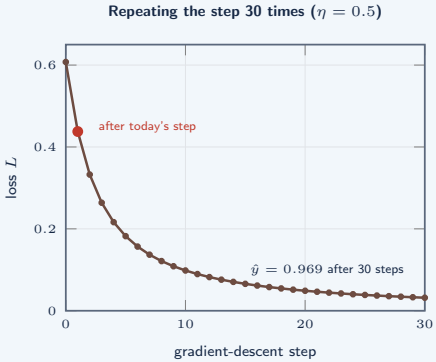
$$W^{(1)} \leftarrow \begin{pmatrix} 0.438956 & -0.180522 \\ 0.273959 & 0.586979 \end{pmatrix}$$

$$b^{(1)} \leftarrow (0.038956, \; -0.026041)$$

$$w^{(2)} \leftarrow (0.830772, \; -0.353017)$$

$$b^{(2)} \leftarrow 0.327650$$

Re-run the forward pass: $\hat{y}$: $0.5447 \rightarrow \mathbf{0.6455}$, L: $0.6075 \rightarrow \mathbf{0.4377}$. The loss fell, so the gradients are right.



1. Forgetting σ' at hidden units. $\delta_j^{(1)} = \delta^{(2)} w_j^{(2)}$ alone is wrong; multiply by $a_j^{(1)}(1 - a_j^{(1)})$.

2. Adding σ' at the output. With BCE the output σ' has already cancelled: $\delta^{(2)} = \hat{y} - y$, not $(\hat{y} - y)\sigma'(z^{(2)})$.

3. Swapping indices. $w_{jk}^{(1)}$ goes *to* unit j *from* input k; its gradient is $\delta_j^{(1)} x_k$, not $\delta_k^{(1)} x_j$.

4. Using updated weights mid-pass. Compute *all* gradients with the old weights first, then update everything together.

5. Sign of the step. Descent is $\theta - \eta \nabla L$. Adding the gradient climbs the loss.

Self-check habit: after any hand calculation, re-run the forward pass with the updated weights. If L did not fall (for a small η), there is an error.

Part 2 Summary — The Four Equations of Backpropagation

OUTPUT ERROR (SIGMOID + BCE)

$$\delta^{(2)} = a^{(2)} - y$$

HIDDEN ERROR

$$\delta_j^{(1)} = \delta^{(2)}\, w_j^{(2)}\, \sigma'(z_j^{(1)})$$

WEIGHT GRADIENTS

$$\frac{\partial L}{\partial w_j^{(2)}} = \delta^{(2)} a_j^{(1)}, \qquad \frac{\partial L}{\partial w_{jk}^{(1)}} = \delta_j^{(1)} x_k$$

BIAS GRADIENTS

$$\frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}, \qquad \frac{\partial L}{\partial b_j^{(1)}} = \delta_j^{(1)}$$

Quick check (answers in the reference text)

1. If y were 0, what would $\delta^{(2)}$ be?
2. Which hidden unit's weights would grow if $w_2^{(2)}$ were $+0.5$?
3. Why is $|\partial L/\partial w_{12}^{(1)}|$ exactly half of $|\partial L/\partial w_{11}^{(1)}|$?
4. What is the largest possible value of $\sigma'(z)$, and why does it matter for deep networks?

Forward: $L = 0.6075$ Backward: 9 gradients One step: $L = 0.4377$

From dense layers to networks that see

- Why a dense layer is a poor fit for a 64×64 image
- Convolution, ReLU and max-pooling computed by hand on a 6×6 image
- Output size $\lfloor (n + 2p - k)/s \rfloor + 1$; padding, stride, channels
- Parameter counting for a full CNN
- LeNet $\rightarrow$ AlexNet $\rightarrow$ VGG $\rightarrow$ ResNet; transfer learning
- Failure modes: adversarial patches, decoys, distribution shift

Assignment 6: single-neuron boundaries and XOR + a ReLU hidden-layer forward pass + a full sigmoid forward and backward pass + one update and the vanishing-gradient argument; bonus: parameter counting and depth.

Preview: SAR target recognition

A 64×64 SAR chip passes through three convolution–pooling blocks (32, 64, 128 filters), global average pooling and a 10-class softmax: **93,962** parameters in total — fewer than a single dense layer on the flattened image would need.

The CNN is trained by *exactly* the backpropagation of today's Part 2. Only the layers are new.